

Product Backlog — Wallet API

Propósito y alcance

Construir en Laravel una API REST de billetera virtual con autenticación, usuarios, cuentas, depósitos, transferencias, CBU de terceros, historial de movimientos y simulación de plazo fijo.

El proyecto se desarrolla en tres sprints. Cada ticket debe completarse y verificarse antes de integrarse en **dev**.

Reglas generales del producto

- La API usa JSON y JWT mediante **php-open-source-saver/jwt-auth**. Las rutas privadas requieren Bearer Token.
- Se mantiene una cuenta por usuario y los importes tienen dos decimales.
- El plazo fijo es una simulación: no descuenta saldo ni crea una inversión real.
- Los listados que pueden crecer deben incluir paginación y ordenamiento.
- Para la demostración se entregará un frontend base que deberá conectarse y verificarse con la API.
- Swagger/OpenAPI documenta y permite probar la API.
- El despliegue público es opcional, se realiza al final y no condiciona la aprobación.
- Todos los endpoints del proyecto utilizan el prefijo **/api/v1**.
- El cacheo, las tarjetas y los pagos online no forman parte de esta entrega.

Criterios generales

- Cumplir los criterios del ticket y validar sus entradas.
- Responder JSON con códigos HTTP apropiados.
- Proteger rutas privadas con JWT y las administrativas también por rol.
- No exponer ni versionar contraseñas, tokens o secretos.
- Mantener los tests existentes en verde.
- Aplicar la Definition of Done sin adelantar requisitos de tickets posteriores.

Sprint 1 — Base, datos y autenticación

WAL-001 — Crear el proyecto y preparar el repositorio

- **Prioridad:** Alta
- **Objetivo:** disponer de un Laravel instalable y ejecutable por todo el equipo.

Criterios de aceptación

- El proyecto funciona localmente y sus migraciones parten de una base vacía.
- El repositorio usa **main**, **dev** y ramas de ticket creadas desde **dev**; los cambios se integran a **dev** por Pull Request.
- **.env** no está versionado y **.env.example** no contiene secretos.
- El README explica dependencias, **.env**, base de datos, migraciones e inicio del proyecto.

Dependencias

Ninguna.

WAL-002 — Crear el modelo de datos inicial

- **Prioridad:** Alta
- **Objetivo:** representar usuarios, cuentas y movimientos con Eloquent y migraciones.

Criterios de aceptación

- Se utiliza `User`; existe `Account` con `id`, `user_id`, `cbu`, `balance` y timestamps.
- Existe `Movement` con `id`, `account_id`, `type`, `amount` y timestamps.
- Los tipos iniciales son `deposit`, `transfer_out` y `transfer_in`.
- Cada usuario tiene exactamente una cuenta; `accounts.user_id` y `cbu` son únicos.
- Están definidas las relaciones User–Account y Account–Movement en ambos sentidos.
- El saldo inicial es `0.00`; saldo y monto usan un decimal apropiado.
- Las migraciones parten de una base vacía y hay factory o seeder de usuarios y cuentas.

Dependencias

WAL-001.

WAL-003 — Registrar un usuario

- **Prioridad:** Alta
- **Historia:** como visitante, quiero registrarme para utilizar la Wallet.

Criterios de aceptación

- Existe `POST /api/v1/auth/register`.
- Recibe como mínimo `name`, `email` y `password` y los valida mediante Form Request.
- El email es válido y único; la contraseña se guarda con hash y no aparece en la respuesta.
- Se crea exactamente una cuenta con CBU único y saldo `0.00`.
- Un registro válido devuelve JSON y `201`; datos inválidos devuelven `422` sin registros parciales.

Dependencias

WAL-002.

WAL-004 — Iniciar sesión con JWT

- **Prioridad:** Alta
- **Historia:** como usuario registrado, quiero autenticarme para acceder a rutas privadas.

Criterios de aceptación

- Se instala `php-open-source-saver/jwt-auth` en una versión compatible con Laravel 13.
- Existe `POST /api/v1/auth/login`; recibe y valida `email` y `password`.
- Credenciales válidas devuelven un JWT utilizable como Bearer Token.
- Credenciales incorrectas devuelven `401` JSON y nunca se expone la contraseña.

- El guard `api` utiliza el driver `jwt` provisto por el paquete.
- `User` implementa `PHPOpenSourceSaver\JWTAuth\Contracts\JWTSubject` y sus métodos requeridos.
- `JWT_SECRET` se configura mediante el entorno y nunca se versiona.
- Una ruta protegida devuelve `401` JSON sin token o con token inválido.

Dependencias

WAL-003.

WAL-005 — Consultar el perfil propio

- **Prioridad:** Alta
- **Historia:** como usuario autenticado, quiero consultar mis datos personales.

Criterios de aceptación

- Existe `GET /api/v1/profile`, protegido con JWT.
- Devuelve `id`, `name` y `email` del usuario obtenido del token, nunca su contraseña.
- El cliente no envía `user_id` y no puede consultar el perfil privado de otro usuario.
- Sin token o con token inválido devuelve `401` JSON.

Dependencias

WAL-004.

WAL-006 — Consultar la cuenta y el saldo propios

- **Prioridad:** Alta
- **Historia:** como usuario autenticado, quiero conocer mi CBU y saldo.

Criterios de aceptación

- Existe `GET /api/v1/account`, protegido con JWT.
- La cuenta se obtiene desde el usuario autenticado; no requiere `user_id` ni `account_id`.
- Devuelve `cbu` y `balance` con dos decimales.
- Cada usuario sólo ve su cuenta; sin token o con token inválido devuelve `401` JSON.

Dependencias

WAL-002 y WAL-004.

Sprint 2 — Operaciones de la Wallet

WAL-007 — Depositar dinero

- **Prioridad:** Alta
- **Historia:** como usuario autenticado, quiero depositar dinero en mi cuenta.

Criterios de aceptación

- Existe `POST /api/v1/deposits`, protegido con JWT.
- Un Form Request exige `amount` numérico y mayor que `0`.
- Opera sólo sobre la cuenta autenticada y aumenta exactamente su saldo.
- Crea exactamente un movimiento `deposit` con la misma cuenta y monto.
- Devuelve el nuevo saldo con dos decimales.
- Un rechazo `422` no cambia saldo ni movimientos.
- `account_id` o `user_id` enviados no permiten elegir otra cuenta.

Dependencias

WAL-006.

WAL-008 — Transferir dinero a otra cuenta

- **Prioridad:** Alta
- **Historia:** como usuario autenticado, quiero transferir dinero a otro CBU.

Criterios de aceptación

- Existe `POST /api/v1/transfers`, protegido con JWT.
- Recibe y valida `destination_cbu` y `amount` mayor que `0`.
- El origen es siempre la cuenta autenticada; el destino existe y no es el CBU propio.
- Saldo insuficiente devuelve `422` sin cambios.
- Una transferencia válida descuenta y acredita exactamente el mismo monto.
- Crea `transfer_out` en origen y `transfer_in` en destino, identificando el CBU contraparte.
- Saldos y movimientos se actualizan dentro de `DB::transaction()`; un fallo no deja cambios parciales.
- Devuelve confirmación y saldo actualizado del emisor.

Dependencias

WAL-007.

WAL-009 — Consultar movimientos con paginación y ordenamiento

- **Prioridad:** Alta
- **Historia:** como usuario autenticado, quiero revisar mi historial por páginas.

Criterios de aceptación

- Existe `GET /api/v1/movements`, protegido con JWT.
- Sólo devuelve movimientos de la cuenta autenticada.
- Cada elemento incluye tipo, monto, fecha y CBU contraparte cuando corresponde.
- Incluye operaciones exitosas; las rechazadas no aparecen.
- La respuesta está paginada y aporta metadatos o enlaces de navegación.
- Utiliza la estructura de paginación nativa de Laravel, con 15 elementos por defecto y un máximo de 100 por página.
- Permite ordenar por fecha ascendente o descendente; por defecto muestra lo más reciente primero.
- Página, cantidad y orden se validan y tienen límites razonables.
- No expone información privada de otros usuarios.

Dependencias

WAL-007 y WAL-008.

WAL-010 — Guardar un CBU de terceros

- **Prioridad:** Alta
- **Historia:** como usuario autenticado, quiero guardar cuentas de destinatarios frecuentes.

Criterios de aceptación

- Existe una relación persistente entre usuario y cuentas de terceros guardadas.
- Existe **POST** `/api/v1/cbu/{cbu}/users/{idUser}`, conservando la estructura indicada por el caso bajo el prefijo versionado.
- `{cbu}` representa el número CBU, no el ID interno de la cuenta.
- La ruta requiere JWT y sólo el propio usuario autenticado puede modificar su lista.
- El CBU corresponde a una cuenta existente, no es el propio y no puede duplicarse para el mismo usuario.
- Un dato inválido no modifica la lista y la respuesta es JSON.

Dependencias

WAL-002, WAL-004 y WAL-008.

WAL-011 — Listar y remover CBU de terceros

- **Prioridad:** Alta
- **Historia:** como usuario autenticado, quiero consultar y quitar destinatarios guardados.

Criterios de aceptación

- Hay un endpoint privado que lista sólo los CBU guardados por el usuario autenticado.
- La lista identifica como mínimo CBU y titular o nombre disponible.
- Existe **DELETE** `/api/v1/cbu/{cbu}/users/{idUser}`, donde `{cbu}` representa el número CBU.
- Un ID ajeno no permite listar ni remover relaciones de otro usuario.
- Remover una relación no elimina la cuenta ni al usuario tercero.
- Las respuestas y errores son JSON.

Dependencias

WAL-010.

WAL-012 — Actualizar o eliminar el perfil propio

- **Prioridad:** Media
- **Historia:** como usuario autenticado, quiero actualizar mis datos personales o eliminar mi perfil.

Criterios de aceptación

- Existe **PUT** o **PATCH** `/api/v1/profile`, protegido con JWT.

- Permite actualizar **name**, **email**, **age** e **image** mediante Form Request; imagen y edad pueden ser opcionales.
- **age**, cuando se informa, debe ser un entero entre 1 y 120.
- **image**, cuando se informa, debe ser un archivo JPG, PNG o WebP de hasta 2 MB.
- El email es válido y único, exceptuando al usuario actual.
- Sólo modifica al usuario autenticado; un **user_id** ajeno es ignorado o rechazado.
- Devuelve el perfil sin contraseña; un **422** no deja cambios parciales.
- Existe **DELETE /api/v1/profile** para eliminar al usuario autenticado.
- La eliminación define y documenta qué sucede con su cuenta, movimientos y CBU guardados mediante las claves foráneas.

Dependencias

WAL-005.

WAL-013 — Completar datos de cuenta y movimientos

- **Prioridad:** Media
- **Objetivo:** incorporar tipo de cuenta, moneda y CBU asociado solicitados por el caso.

Criterios de aceptación

- **Account** contiene tipo de cuenta (**savings** o **checking**) y moneda (**ARS** o **USD**).
- **User** puede almacenar edad e imagen; ambos campos admiten ausencia para no romper registros existentes.
- Las cuentas existentes reciben valores predeterminados válidos.
- La consulta de cuenta devuelve esos campos.
- Las transferencias conservan el CBU de origen o destino correspondiente.
- Importes y saldos pertenecen a la moneda de la cuenta; no se exige conversión.
- Las migraciones funcionan desde una base vacía y sobre datos de prueba.

Dependencias

WAL-002 y WAL-008.

WAL-014 — Simular un plazo fijo

- **Prioridad:** Alta
- **Historia:** como usuario autenticado, quiero conocer el rendimiento estimado de un plazo fijo.

Criterios de aceptación

- Existe **POST /api/v1/investments/fixed-term/simulate**, protegido con JWT.
- Recibe monto y plazo o fecha futura; un Form Request valida ambos.
- La simulación utiliza interés simple, TNA del 30%, base de 365 días y un plazo permitido de 30 a 365 días.
- La tasa está guardada en configuración, está documentada y no puede ser elegida libremente por el cliente.
- Devuelve fecha de creación, finalización, monto invertido, interés ganado y total a cobrar.

- Los valores monetarios tienen dos decimales y la fórmula/unidad de tasa está explicada.
- Datos inválidos devuelven 422 JSON.
- La simulación no cambia el saldo ni crea movimientos.

Dependencias

WAL-004 y WAL-013.

Sprint 3 — Administración, calidad y entrega

WAL-015 — Incorporar roles de usuario y administrador

- **Prioridad:** Alta
- **Objetivo:** distinguir usuarios comunes de administradores.

Criterios de aceptación

- Existen los roles `user` y `admin`, o equivalentes documentados.
- El registro público asigna el rol común y no permite autoasignarse administrador.
- Un seeder crea un administrador de prueba sin publicar credenciales reales.
- Middleware o autorización equivalente protege rutas administrativas.
- Un usuario común recibe 403 JSON y un administrador autenticado puede acceder.

Dependencias

WAL-004.

WAL-016 — Administrar usuarios

- **Prioridad:** Alta
- **Historia:** como administrador, quiero crear, listar, consultar, actualizar y eliminar usuarios.

Criterios de aceptación

- Hay endpoints CRUD administrativos, todos protegidos por JWT y rol.
- Crear valida datos, hashea contraseña y crea la cuenta del usuario.
- Actualizar valida email único y nunca expone la contraseña.
- El listado está paginado y admite ordenamiento básico.
- Al eliminar se respeta el comportamiento documentado de cuenta y movimientos según las claves foráneas.
- Un usuario común no puede ejecutar estas operaciones.

Dependencias

WAL-015.

WAL-017 — Administrar cuentas

- **Prioridad:** Media
- **Historia:** como administrador, quiero consultar y mantener cuentas.

Criterios de aceptación

- Hay endpoints CRUD administrativos, protegidos por JWT y rol.
- El listado está paginado y admite ordenamiento básico.
- Se validan unicidad de usuario y CBU, tipo de cuenta, moneda y saldo.
- Sólo se crea una cuenta para un usuario que todavía no tenga una.
- Actualizar no reasigna accidentalmente la cuenta a otro usuario.
- La eliminación respeta relaciones y devuelve un error claro cuando no es válida.
- Estas acciones no crean depósitos o transferencias ficticios.

Dependencias

WAL-013 y WAL-015.

WAL-018 — Administrar transacciones o movimientos

- **Prioridad:** Alta
- **Historia:** como administrador, quiero mantener el historial de movimientos.

Criterios de aceptación

- Hay endpoints CRUD administrativos, protegidos por JWT y rol.
- El listado está paginado, ordenado y puede filtrarse por cuenta o usuario.
- Crear o actualizar valida cuenta, tipo, monto y CBU contraparte.
- Guardar o actualizar devuelve la entidad completa con cuenta, moneda y CBU asociado.
- Conforme al caso, editar o eliminar un movimiento modifica el historial y no recalcula automáticamente el saldo.
- Esa limitación queda documentada para no confundir este CRUD con depósitos o transferencias.
- Un usuario común no puede acceder.

Dependencias

WAL-009, WAL-013 y WAL-015.

WAL-019 — Estandarizar paginación, ordenamiento y filtros

- **Prioridad:** Media
- **Objetivo:** recorrer listados grandes de manera consistente.

Criterios de aceptación

- Movimientos, usuarios, cuentas y listados administrativos usan el paginador de Laravel.
- Todos conservan la estructura de paginación nativa de Laravel.
- Se devuelven 15 elementos por defecto y se admite un máximo de 100 por página.
- Campo y dirección de orden sólo aceptan valores permitidos.
- Los filtros no amplían el acceso del usuario autenticado.
- La estructura de metadatos o enlaces es consistente.
- Parámetros inválidos reciben 422 o valores predeterminados documentados de forma uniforme.

Dependencias

WAL-009, WAL-016, WAL-017 y WAL-018.

WAL-020 — Unificar errores HTTP y seguridad básica

- **Prioridad:** Alta
- **Objetivo:** ofrecer errores predecibles y medidas de seguridad concretas.

Criterios de aceptación

- Autenticación, autorización, validación y recurso inexistente responden JSON.
- Toda solicitud bajo `/api/v1`, incluso sin el encabezado `Accept: application/json`, devuelve errores JSON y nunca intenta redirigir hacia una ruta web de login.
- Se usan consistentemente `401`, `403`, `404` y `422`.
- Los errores no revelan contraseñas, tokens, SQL ni detalles internos.
- Se validan entradas y se usa Eloquent o parámetros enlazados.
- Las vistas no insertan contenido sin escapar.
- El login admite como máximo 5 intentos fallidos por minuto por la combinación de email normalizado e IP.
- Al superar el límite, responde `429` JSON e informa cuándo puede reintentarse.
- `.env`, `JWT_SECRET`, tokens y credenciales no están versionados.

Dependencias

WAL-015 a WAL-019.

WAL-021 — Completar y estabilizar la cobertura funcional

- **Prioridad:** Alta
- **Objetivo:** auditar los tests existentes, completar únicamente los casos faltantes y comprobar automáticamente las funciones críticas.

Criterios de aceptación

- Existe una matriz que relaciona los flujos requeridos con los tests que los cubren.
- No se duplican escenarios que ya cuenten con cobertura suficiente.
- Hay tests de registro/login y acceso privado sin token.
- Hay tests de perfil propio, depósito válido e inválido, saldo y movimientos.
- Hay tests de transferencia válida, saldo insuficiente y ausencia de cambios parciales.
- Hay tests de aislamiento entre usuarios para cuenta, movimientos y CBU guardados.
- Hay tests de plazo fijo válido e inválido y se verifica que no cambia el saldo.
- Al menos un test distingue permisos de usuario y administrador.
- Se prueba paginación y ordenamiento de al menos un listado.
- Los tests usan una base de prueba y se ejecutan con el comando documentado.
- Los datos relevantes son deterministas y la suite puede ejecutarse en orden aleatorio sin depender de una ejecución anterior.

Dependencias

WAL-003 a WAL-020.

WAL-022 — Documentar la API con Swagger/OpenAPI

- **Prioridad:** Alta
- **Historia:** como consumidor, quiero conocer y probar los endpoints.

Criterios de aceptación

- Swagger se genera sin errores y Swagger UI está disponible en una URL documentada.
- Incluye autenticación, perfil, cuenta, depósitos, transferencias, CBU de terceros, movimientos, plazo fijo y administración.
- Cada operación indica método, URL, parámetros, cuerpo y respuesta exitosa.
- Documenta errores relevantes 401, 403, 404 y 422.
- Explica Bearer JWT y permite probar rutas protegidas.
- Refleja paginación y ordenamiento.
- No exige componentes OpenAPI avanzados si una definición sencilla es suficiente.

Dependencias

WAL-003 a WAL-020.

WAL-023 — Integrar el frontend demostrativo provisto por el docente

- **Prioridad:** Media
- **Objetivo:** conectar con la API el frontend base entregado para la demostración.

Criterios de aceptación

- Se recibe una vista Blade o HTML/JavaScript base con instrucciones.
- Consume la API mediante AJAX/JSON.
- Demuestra como mínimo login, perfil, cuenta, depósito, transferencia e historial.
- Usa el token en rutas privadas sin publicarlo en el repositorio.
- Muestra los errores principales de forma comprensible.
- Si una persona intenta abrir una sección privada del frontend sin autenticarse, se la dirige al formulario de login.
- No se requiere desarrollar un frontend desde cero ni incorporar otro framework.
- La API continúa siendo utilizable sin el frontend.

Dependencias

WAL-021 y WAL-022.

WAL-024 — Realizar la revisión final y preparar la demo

- **Prioridad:** Alta
- **Objetivo:** comprobar que el proyecto puede instalarse, probarse y explicarse.

Criterios de aceptación

- El README cubre instalación, `.env`, base, migraciones, seeders, tests, inicio y Swagger.
- Una instalación limpia ejecuta migraciones y seeders sin errores; todos los tests pasan.
- No hay `.env`, contraseñas, tokens ni secretos versionados.
- Rutas privadas y administrativas respetan autenticación y roles.
- La demo usa al menos dos usuarios y un administrador.
- Muestra login, perfil, cuenta, depósito, transferencia, CBU guardados, movimientos paginados y plazo fijo.
- Incluye un error de validación, saldo insuficiente y permiso administrativo denegado.
- Se usa el frontend provisto o Swagger y cada integrante puede explicar su aporte.

Dependencias

WAL-021, WAL-022 y WAL-023.

WAL-025 — Desplegar una demo pública (opcional)

- **Prioridad:** Baja
- **Carácter:** Opcional; no bloquea la aprobación ni la demo local.
- **Objetivo:** publicar la aplicación si el tiempo y la infraestructura lo permiten.

Criterios de aceptación

- Plataforma y pasos de despliegue están documentados.
- Los secretos son variables de entorno y no se versionan.
- La base se prepara mediante migraciones.
- Aplicación y Swagger abren desde la URL publicada.
- Se prueba login y una ruta protegida.
- Si no se implementa, WAL-024 puede aprobarse mediante ejecución local.

Dependencias

WAL-024.

Resumen

Etapas	Tickets	Resultado esperado
Sprint 1	WAL-001 a WAL-006	Proyecto, datos, registro, JWT, perfil y cuenta.
Sprint 2	WAL-007 a WAL-014	Depósitos, transferencias, CBU de terceros, movimientos paginados, edición de perfil y plazo fijo.
Sprint 3	WAL-015 a WAL-024	Roles, administración, seguridad, tests, Swagger, frontend provisto y demo.
Opcional	WAL-025	Demo pública si hay tiempo e infraestructura.

Funcionalidades que no forman parte de la entrega

- Tarjetas físicas o virtuales y pagos online.
- Múltiples cuentas por usuario y conversión real de monedas.
- Compra, renovación o cancelación real de plazo fijo y tasas externas en tiempo real.
- Cacheo de respuestas.
- Arquitectura hexagonal, CQRS, Repository Pattern, Events, Jobs o microservicios.
- CI/CD, monitoreo, observabilidad, pentesting y porcentaje mínimo de cobertura.
- Desarrollo de un frontend desde cero.
- Despliegue público como condición de aprobación.